When Self-Healing Hurts: Reproducing Signal-Based Health Monitoring for Kubernetes and Fixing Probe-Induced Restart Cascades

Deliverable 02 – Base-Article Implementation and Real-Gap Finding

M. Yasir Suhail
Department of Computer Science
FAST-NUCES, Islamabad
Section A, Group G18

M. Aashir Sheraz
Department of Computer Science
FAST-NUCES, Islamabad
Section A, Group G18

***Abstract*—Kubernetes restarts a container when its liveness probe fails. Our base paper, Roberts *et al.* [1], shows that this poll-based monitoring is slow and proposes signal-based monitoring (SKI) instead. In this deliverable we (1) re-implement the core of that paper on stock Kubernetes, without the authors' modified kubelet, and reproduce its main results; (2) use the reproduction to locate a real gap: the paper tests one pod under light load, a setting in which a probe can never cause a cascade, and neither probes nor SKI look at all replicas together before restarting; (3) propose a method with two parts: *progress-based liveness* (A), which judges a container by whether it still finishes work rather than by whether it answers quickly, and a *cluster-aware restart governor* (B), which holds restarts back when many replicas fail at the same time; and (4) evaluate it with an ablation. A calibrated simulator reproduces the paper's detection times within about 1 s. It shows that at 90 % load the paper's "fast probe" baseline cuts successful requests from 100.0 % to 58.9 % even though nothing is broken. After a 60 s load spike it leaves the service at 4.6 % success; with no probe at all the service recovers to 99.3 %. A+B removes all of these false restarts, keeps 92.2 % success during a dependency stall (baseline: 66.0 %), and still detects a silently hung replica within 5.4 s. All code, scripts and a one-command Kubernetes testbed for an 8 GB laptop are provided. Testbed tables in this report fill in automatically from the raw runs.**

***Index Terms*—Kubernetes, liveness probe, self-healing, failure detection, overload, restart cascade, metastable failure, reproducibility**

I. Introduction

Self-healing is one of the main reasons people run services on Kubernetes. The kubelet sends a *liveness probe* (an HTTP request, TCP connect or command) to each container every few seconds. If the container fails a number of probes in a row, the kubelet kills and restarts it [2]. The hidden assumption is simple: *a container that does not answer in time is broken.*

In Cutoff 1 we argued that this assumption is wrong under overload. A busy container is slow, not broken. If its health endpoint waits in the same queue as real requests, the probe times out and the healthy container is killed. Its traffic moves to the other replicas, which then slow down and fail their probes too. The repair mechanism becomes the fault.

This deliverable follows the workflow the instructor asked for. We *reproduce* the base paper, *analyse* it, find a *defensible gap*, then *design, implement and evaluate* our own method with an ablation study. Our contributions are:

1) **A decision and a reproducibility check** for the base paper (Sec. II). We keep Roberts *et al.* and explain why, including what is missing from it.
2) **A working re-implementation of its core method on stock Kubernetes** (Sec. V). The paper's SKI needs a modified kubelet that is not public. We rebuild the same signal path with a log sidecar, a shared PID namespace and a readiness gate. We also give a calibrated simulator that reproduces the paper's Tables IV–VI.
3) **An explanation for the paper's own unexplained result.** The extra "fast probe" restarts come from a race between start-up and the first liveness probe (Sec. VI-B).
4) **A defensible research gap**, shown with evidence: the cascade appears only when there is more than one replica, which the paper never tests (Sec. VIII).
5) **A proposed method** (progress-based liveness + cluster-aware restart governor), its code, and an **ablation** over four fault and load scenarios (Secs. IX–XI).

II. Selecting the Base Paper: Does It Fit?

A. The selected paper

- **Title:** Signalling Health for Improved Kubernetes Microservice Availability [1]
- **Authors:** Jacob Roberts, Blair Archibald, Phil Trinder (University of Glasgow)
- **Venue / year:** arXiv preprint 2507.02158 (cs.DC), July 2025
- **Paper link:** https://arxiv.org/abs/2507.02158
- **GitHub / project link:** none published. The paper links only to SockShop [3], Locust [4], Kubernetes and CRI-O.

TABLE I
REPRODUCIBILITY CHECK OF THE BASE PAPER

Item	Available	What we did
Official repository	No	Re-implemented everything (Go + YAML)
Source code (SKI)	No	Built an SKI emulation on stock K8s
Modified kubelet	No	Not needed: sidecar + PID ns + readiness gate
Experiment scripts	No	Wrote `experiments/*.sh`
Benchmark / data	Yes	SockShop catalogue + catalogue-db images
Configuration	Partly	Probe table given; `timeoutSeconds` and startup-probe values missing
Software versions	Yes	K8s 1.27.8, CRI-O 1.27.1; we use K8s 1.27.13 (kind)
Hardware	Yes	1 VM, 8 vCPU, 32 GB; we use a laptop
Install / run guide	No	`docs/GUIDE.md`
Reported results	Yes	Tables III–VI used as targets

B. Reproducibility check

Table I goes through the instructor's checklist. The paper is weak on released artefacts. Its *core method*, however, is small and fully specified. It consists of four probe settings (Table II of the paper), a closed-form detection-time model, one public benchmark service, one load generator and one fault at a fixed time. That is exactly the "core methodology" the instructor asks us to implement.

C. Alternatives considered

We looked for a better-documented base paper in the same area before deciding. Flora *et al.* [5] study probe *false negatives* under software ageing. They publish no code, and their topic is the opposite failure mode to ours. Chen *et al.* [6] inject node, pod and network faults at scale, but they do not study probes and release no code. Vayghan *et al.* [7] treat Kubernetes as an availability manager, but their work is older, uses no probes under load and has no public artefact. None of these is easier to reproduce, and none of them touches our research question.

Decision. We keep Roberts *et al.* for three reasons.

- It is the closest work to our question.
- It *observes* the very phenomenon we study but cannot explain it.
- Its future-work section predicts our result: "we expect PCM to erroneously detect high network latency as container failure".

Its two weaknesses are that it is a preprint and that it has no code. We handle both openly by publishing a full re-implementation.

D. A note on "datasets" and modality

This is a *systems* paper, not a machine-learning paper. There is no training data and no text, image or audio modality. The equivalent of a dataset is the *benchmark service plus the workload*. Section IV documents these the way the instructor asked for datasets.

III. UNDERSTANDING THE BASE PAPER

Problem. Kubernetes notices failures by *polling* (poll-based container monitoring, PCM). Polling must be tuned by hand. Detection takes several probe periods. Badly tuned probes can lower availability.

Why it matters. Every second between a failure and its repair is lost availability. Default probe settings in benchmarks such as SockShop take minutes to recover a pod.

Proposed solution. In *signal-based* monitoring (SCM), the container *tells* the orchestrator when its state changes. In SKI, the Kubernetes prototype, a sidecar follows the container's log through the API server. A matching log line makes the sidecar notify a *modified kubelet* over a UNIX socket. The kubelet then marks the container Ready or restarts it at once.

Analytical model. Detection time for polling is $T_{l,p} \approx (N_l - \frac{1}{2})I_l + L_l$, where N_l is the failure threshold, I_l the period and L_l the probe latency. For signals it is $T_{l,s} \approx L_s$.

Cloud component. Self-healing in a container orchestrator: the kubelet's monitor–decide–act loop on each node.

Benchmark and baselines. SockShop's `catalogue` service (one pod) backed by `catalogue-db`. The two baselines are the SockShop default probes (DP: 3 failures, every 3 s, delays 300/180 s) and fast probes (FP: 1 failure, every 1 s, delay 1 s).

Metrics. Time to queue a restart, time to start the restart, total/failed/HTTP-500 requests, cumulative Ready pods, CPU and memory overhead.

Settings. One VM (8 vCPU, 32 GB), single-node Kubernetes 1.27.8, CRI-O 1.27.1, one pod per service, Locust at $\leq$100 req/s with a 0.5 s timeout, a 6-minute warm-up, failure at 60 s, 290 s windows and 30 runs per configuration.

Assumptions. Failures are clean (a process dies) and the container can *say* it failed by writing a log line. There is only one replica.

Limitations the authors state. A simple failure model, a small benchmark and a single node with one pod. SKI does not aggregate status across pods. Its log fetching may not scale. "It remains unclear why most restarts under FP occur after the failure at 60 s."

Missing or unclear details. No code. No `timeoutSeconds` or startup-probe values. Locust user count, spawn rate and wait time are not given, and neither are the log patterns SKI matches. The "catalogue bug" adds restarts that are not part of the experiment.

IV. BENCHMARK, WORKLOADS AND DATA

Table II lists every workload used, in the format the instructor asked for. Nothing is "trained", so there is no train/validation/test split. Instead, every configuration is *repeated* with different random seeds. Simulator runs are fully seeded and therefore exactly repeatable.

TABLE II
WORKLOADS (THE "DATASETS" OF THIS SYSTEMS STUDY)

Name	Source	Size	Characteristics	Pre-processing / use
SockShop `catalogue` 0.3.5 + `catalogue-db` 0.3.0	Docker Hub, [3]	2 images, 9 catalogue items (MySQL)	Go REST service; `/health` pings the DB; Ready in ≈2 s after start	Unchanged image; run under a shell wrapper exactly as the paper does (liveness test)
Paper workload (R)	Re-created from [1]	100 users × 1 req/s, 290 s	Closed model, 0.5 s timeout, `GET /catalogue`	Our Go generator in *closed* mode replaces Locust
G18 service under test	This work (`src/cmd/sut`)	6 replicas, 4 workers each	CPU-bound `/work` behind a FIFO pool shared with `/healthz`	CPU limit 200m; fixed CPU work per request
Overload workloads (O, P)	This work	60–120 % of measured capacity, 240 s	Open model, Poisson arrivals, 2 s timeout; P = 70 %→150 % spike for 60 s	Rates scaled by the calibrated capacity (Exp. C)
Fault workloads (H, D)	This work	70 % load, 240 s	H: one replica hangs silently at 60 s; D: shared dependency stalls 20 s	Faults injected through admin endpoints
Raw results	Generated	≈300 KB per run	Per-second request counts, 1 Hz pod state, events, logs	`analysis/analyze_testbed.py`

V. REPRODUCTION SETUP

A. Two complementary instruments

We reproduce the paper in two ways.

(1) Kubernetes testbed (`experiments/`). This is a kind cluster [8] pinned to Kubernetes 1.27.13, the closest 1.27 release that kind ships. It runs on one laptop, with the same single-node, single-pod design as the paper. Each run gets its own namespace, so events from one run cannot leak into the next. Load comes from a generator *inside* the cluster, so it goes through the normal Service load balancing.

(2) Discrete-event simulator (`sim/probesim.py`, about 900 lines of Python). It models the same mechanisms one by one:

- a FIFO worker pool and a CPU limit;
- kubelet probe workers;
- the kubelet restart back-off, as coded in Kubernetes 1.27 (`doBackOff`);
- endpoint propagation;
- open and closed clients;
- the faults;
- both proposed components.

The simulator lets us run thousands of seeded runs in minutes and sweep settings a laptop cannot sweep. The testbed checks that the effects are real.

B. SKI on stock Kubernetes

Table III maps each part of SKI to our emulation. The signal path is the same: the sidecar follows the log, matches a pattern and acts. Only the last step differs, because we may not modify the kubelet.

TABLE III
SKI (PAPER) VS. OUR EMULATION

Paper (modified kubelet)	Ours (stock Kubernetes 1.27)
Sidecar follows log via API	Same (`src/cmd/ski`)
Substring match	Same (`G18-FAILURE`, `transport=HTTP`)
Kubelet queues restart	SIGKILL via shared PID namespace; kubelet restarts container
2 s termination grace	2 s delay before the kill
Kubelet marks Ready	Sidecar sets a readiness-gate condition [9]

C. Environment

Hardware. The paper used a VM with 8 vCPU and 32 GB. Our target is an HP 348 laptop (Intel Core i5, 4 cores/8 threads, 8 GB). The testbed records the machine it actually ran on in `results/testbed/environment.txt`. For the run used here: CPU *pending*, *pending* threads, *pending* GB RAM, OS *pending*, Docker *pending*.

Cloud resources. None. The study is designed to run on one laptop. CPU limits make each pod saturate before the machine does.

Software. Kubernetes 1.27.13 (kind v0.23.0, containerd), Go 1.22, Python 3.10+ with matplotlib. The simulator and analysis need no other packages.

Key settings are kept in `experiments/config.env`. They include 100 users at 1 req/s with a 0.5 s timeout, failure at 60 s, a 290 s window, the paper's probe table, 6 replicas, 4 workers, a 200m CPU limit and stall window $W = 5$ s.

D. Deviations from the original experiment

1) **containerd instead of CRI-O, and K8s 1.27.13 instead of 1.27.8.** The probe and restart code paths are the same.
2) **Locust is replaced by our Go generator in closed mode.** The generator is lighter and is not needed on the host. The paper does not give its Locust settings.
3) **Shorter warm-up for FP and SKI** (120 s instead of 360 s). Liveness starts after 1 s, so the warm-up only has to let the pod settle. DP keeps 330 s because its liveness delay is 300 s.
4) **No "catalogue bug".** The database is started first and shared, so the start-up restart the paper describes does not happen. The expected number of restarts is therefore 1 rather than 2.
5) **Fewer repetitions** on the testbed: 3 runs per configuration ("quick" profile) or 5 ("full"), not 30. The simulator uses 30.

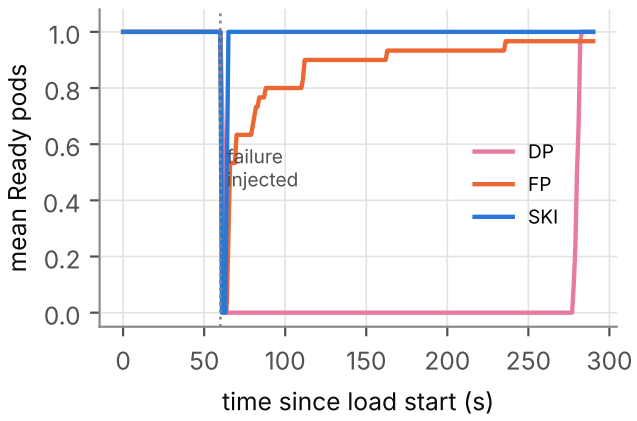


Fig. 1. Mean number of Ready catalogue pods (simulator, 30 runs). This is the counterpart of the paper's Fig. 7. DP needs 180 s of readiness delay after the restart. FP sometimes enters a restart loop. SKI recovers in about 2 s.

VI. Reproduction Results

Table IV puts the paper, the simulator and the testbed side by side.

A. What was reproduced

Detection times. The simulator gets within about 1 s of every detection time in the paper:

- DP: 7.9 s vs. 8.7 s to queue, 38.1 s vs. 38.9 s to start;
- FP: 0.8 s vs. 0.7 s to queue, 3.0 s vs. 2.9 s to start;
- SKI: 0.1 s vs. 0.1 s to queue, 2.3 s vs. 2.2 s to start.

Successful requests. The counts match closely: 6902 vs. 6890 (DP), 25 961 vs. 26 900 (FP) and 28 506 vs. 27 989 (SKI).

Availability. Cumulative Ready pods are 73/263/288 against the paper's 80/274/286. The ranking DP $\ll$ FP $<$ SKI and the "3.4–3.6 times better than DP" result both hold.

B. Explaining the paper's unexplained FP restarts

The paper reports 3 ± 1 restarts under FP (up to 6) and states that "it remains unclear why most restarts under FP occur after the failure at 60 s." Our model gives an explanation.

FP starts liveness probing 1 s after the container starts, with a threshold of 1. The catalogue needs about 1.5 s to start listening. The first probe after a restart therefore often lands *before* the service is up. It fails, and the new container is killed again.

The kubelet's back-off makes the first two such restarts free. From the third restart it adds 10 s, then 20 s, and so on. This also explains the paper's remark that back-off "was applied inconsistently". With this one mechanism the simulator produces 1–6 restarts after the failure (mean 1.9 over an expected 1). Extra restarts occur in 47 % of runs, and almost all of them come right after the induced failure, just as the paper observed.

On the testbed this can be checked directly. The kubelet events for these restarts read "Liveness probe failed: …connection refused", and they appear within 2 s of a container start.

C. Differences and possible reasons

1) **Total and failed request counts.** The paper's DP arm sends only 7600 of the 29 000 possible requests. Its Locust users therefore *block* while no endpoint exists, instead of failing fast. Our generator fails fast, so we count those requests as failed. *Successful* requests are the comparable quantity, and they match.
2) **DP queue time: 7.9 vs. 8.7 s.** The paper's own model also predicts about 7.7 s. The authors attribute the extra second to failure timing inside the probe period. We see the same gap, which points to kubelet scheduling slack that neither model includes.
3) **FP availability: 263 vs. 274.** This depends strongly on the catalogue start-up time and on back-off timing. We calibrated the start-up time to the paper's "Ready in about 2 s". A slightly faster start gives fewer restart loops.
4) **Hardware and runtime.** A laptop instead of an 8 vCPU VM, and containerd instead of CRI-O, change start-up and exec latencies by tens to hundreds of milliseconds.

D. Overhead (paper's Exp. 1)

The paper reports that SKI adds 0.04 vCPU and 9 MiB per pod. On the testbed we read cgroup counters during the first 55 s of load:

- catalogue alone: *pending* millicores, *pending* MiB;
- SKI sidecar: *pending* millicores, *pending* MiB.

TABLE IV
REPRODUCTION OF ROBERTS ET AL. (TABLES IV–VI OF THE PAPER). MEAN ± STD. SIMULATOR: 30 SEEDED RUNS PER ARM. TESTBED: *pending* RUNS PER ARM ON THE LAPTOP.

Arm	Metric	Original paper	Reproduced (simulator)	Reproduced (testbed)
DP	Time to queue restart (s)	8.7 ± 0.5	7.9 ± 0.9	*pending*
	Time to start restart (s)	38.9 ± 0.3	38.1 ± 0.9	*pending*
	Successful requests	6890	6902	*pending*
	Restarts after failure	1	1.0 ± 0.0	*pending*
	Cumulative Ready pods (max 291)	80	73	*pending*
FP	Time to queue restart (s)	0.7 ± 0.5	0.8 ± 0.3	*pending*
	Time to start restart (s)	2.9 ± 0.5	3.0 ± 0.3	*pending*
	Successful requests	26900	25961	*pending*
	Restarts after failure	3 ± 1[†]	1.9 ± 1.2	*pending*
	Cumulative Ready pods (max 291)	274	263	*pending*
SKI	Time to queue restart (s)	0.1 ± 0.3	0.1 ± 0.0	*pending*
	Time to start restart (s)	2.2 ± 0.4	2.3 ± 0.0	*pending*
	Successful requests	27989	28506	*pending*
	Restarts after failure	1	1.0 ± 0.0	*pending*
	Cumulative Ready pods (max 291)	286	288	*pending*

[†]The paper expects 2 restarts under FP: one from its start-up "catalogue bug" and one from the failure. It observes 3±1 (range 2–6). Our setup has no start-up bug, so we expect 1. "Successful requests" = total − failed in the paper's Table V. We compare successes because the paper's Locust users stop sending while the service is down, so its *total* counts are not comparable (Sec. VI-C).

VII. PROBLEMS ENCOUNTERED

Missing resources. The paper releases no code and no modified kubelet, and leaves the timeout and startup-probe values undocumented (Sec. II).

Installation. The paper's own environment cannot be installed, because the modified kubelet is private. Docker Hub rate limits can stop `kind` pulls, so the setup script pre-loads every image into the node once. The catalogue cannot bind port 80 as a non-root user without extra capabilities, so we use its own default port, 8080.

Code problems found in our Cutoff-1 code. Our review found three bugs. Each would have invalidated the study:

1) `kubectl port-forward svc/sut` tunnels *all* traffic to *one* pod. The other five replicas got no load, so load redistribution, the mechanism we study, could not happen. The forward also died whenever that pod restarted. *Fix:* an in-cluster load generator.
2) `burn()` looped until a *wall-clock* deadline. A throttled container therefore did *less* work rather than taking longer, and the 200m CPU limit never limited capacity. *Fix:* a fixed amount of CPU work per request.
3) Restart attribution read *all* `Unhealthy` events, which mixes readiness and liveness failures. Events from earlier runs in the same namespace leaked into later ones. *Fix:* one namespace per run, and only events reading "failed liveness probe, will be restarted" are counted.

Computational limits. A laptop cannot run 30 repetitions of every arm, so the testbed uses 3–5 repetitions and the simulator provides the statistics.

Scope change from Cutoff 1. Cutoff 1 planned loads of 50, 100 and 150 %. With an open workload, a FIFO queue and a 2 s client timeout, *every* arm collapses above about 110 %, because almost every request times out while it waits (Fig. 5, [10]). The informative region is just below saturation and during short spikes, so the testbed now uses 90 % plus a spike experiment. The full 50–150 % grid is still run in the simulator (Sec. XI-G).

VIII. THE RESEARCH GAP

The reproduction and the analysis above give three concrete weaknesses of the base paper. We did not pick them because a new technique looked interesting. Each one comes from the paper's own design or text.

G1 – A single pod hides the failure mode. With one replica there is nowhere for load to move, so a probe-induced restart cannot spread. Fig. 2 tests this directly. We keep the load at 95 % of capacity and change only the number of replicas. With FP and *one* replica, the success rate is 93.5 %, close to running with no probe. With two replicas it falls to 49.5 %, and from four replicas up it falls to about 20 %. The paper's evaluation is at $N = 1$, where the problem is invisible by construction.

G2 – "Slow" is treated as "dead". PCM judges a container by *response time*. When the health check shares resources with real work, as `catalogue`'s `/health` shares the database, response time grows with load. The authors expect PCM "to erroneously detect high latency as container failure" but never test it.

SKI avoids that particular error, but it has the opposite problem: it needs the container to *report* its failure in a log line. A silently hung process writes nothing, so SKI never restarts it. In the hang experiment (H, Sec. XI) SKI would behave exactly like NONE.

G3 – No aggregation across replicas. Both the kubelet and SKI decide per container. The paper names this as a limitation: SKI "does not aggregate pod status before restarting". When the cause is *shared*, such as overload or a slow dependency, every replica fails at once, and each one is restarted, removing capacity exactly when it is needed.

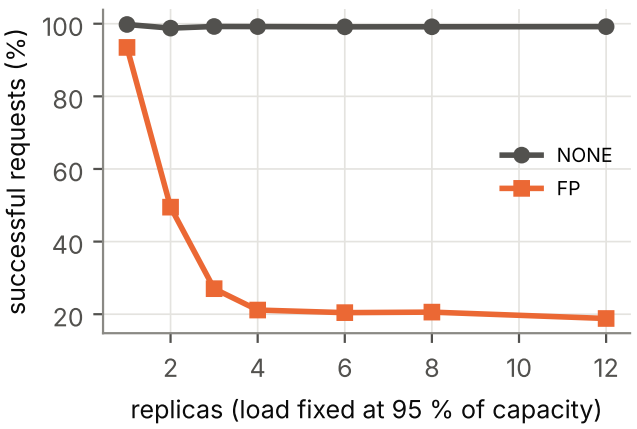


Fig. 2. Evidence for G1 (simulator, 10 runs per point, 95 % load). With one replica the fast probe does almost no harm. With more replicas it produces a cascade.

Gap statement. Existing health monitoring for Kubernetes, whether poll-based or signal-based, (i) cannot tell a slow container from a dead one without help from the application, and (ii) decides restarts one container at a time. In a replicated service under overload or a shared brown-out, this turns self-healing into a source of failure. The base paper's single-pod, light-load evaluation cannot reveal this, and it remains untested.

Research questions for the rest of the project.

- *RQ1* (Cutoff 1): under overload, at which probe settings does self-healing reduce availability compared with no probe?
- *RQ2:* can a liveness signal tell slow from dead well enough to remove these false restarts while still catching real hangs?
- *RQ3:* does deciding restarts across replicas prevent cascades when the cause is shared?

IX. Proposed Methodology

This section is written so that it can move into the final paper with few changes. Fig. 3 gives the overview.

A. Problem definition

A Deployment has N replicas. Replica i serves requests through a bounded pool and keeps a queue of pending work $q_i(t)$. A liveness mechanism is a function that, at each check time t, decides whether to restart replica i. We want a mechanism that:

1) never restarts a replica that is *making progress*, however slowly (no false positives under overload);
2) restarts a replica that has *stopped making progress* within a bounded time (no false negatives for hangs);
3) does not restart many replicas at once for a cause that a restart cannot fix.

Algorithm 1 One governor tick

$E \leftarrow$ running replicas older than grace G (10 s)
$S \leftarrow \{i \in E : \text{consecutive failed checks}_i \geq F\}$
if $S = \emptyset$ **then**
  reset hold; **return**
end if
if $|S|/N \geq \theta$ **and** $|S| \geq 2$ **then** ▷ correlated
  if hold not started **then** start hold
  end if
  if hold age $< H$ **then**
    emit event RestartHeld; **return**
  end if
end if
$b \leftarrow \max(1, \lfloor \beta N \rfloor) - |\text{in-flight restarts}|$
restart up to b replicas of S, most failures first
futility: if a replica we restarted fails again within R (30 s) of recovering, set $H \leftarrow \min(2H, H_{\max})$

B. Component A: progress-based liveness (PBL)

Each replica keeps two numbers: the count of pending requests p_i (queued or running) and the time ℓ_i of the last *completed* request. The liveness answer is

$$\text{alive}_i(t) \;=\; \big[p_i(t) = 0\big] \;\vee\; \big[t - \ell_i(t) < W\big] \qquad (1)$$

where W is the *stall window*. Two details make this robust:

- When p_i goes from 0 to 1, ℓ_i is reset to now. An idle container is therefore never "stalled since the last request an hour ago".
- Requests that clients abandon while still queued do *not* count as progress, so a hung pool cannot look alive just because clients keep timing out.

The check is served *outside* the worker pool, so it answers in microseconds whatever the queue length. It is a standard HTTP endpoint used by an *unmodified* kubelet probe (`/livez`, FP timing). It is about 40 lines of Go logic (`src/internal/progress`) and needs one `Begin()` and one `End()` call around request handling.

Equation (1) classifies the three cases correctly:

- an overloaded container keeps completing work, so it is alive;
- a hung container has $p_i > 0$ but completes nothing, so it is dead after W;
- an idle container has $p_i = 0$, so it is alive.

It is the classic heartbeat or watchdog idea from failure-detector theory [11], [12], applied to the container's *work* instead of to a probe's round-trip time.

Why not simply isolate the health check? A health endpoint on its own thread that always returns 200 (the ISO arm) also avoids false positives under overload. It can never detect a hang, though, which is the false-negative problem of Flora *et al.* [5]. A keeps hang detection, as the H experiment shows.

C. Component B: cluster-aware restart governor (CRG)

The governor replaces the kubelet's liveness decision; the Deployment then has no `livenessProbe`. Every P=1 s it checks every replica (`/livez` with A, `/healthz` without) and applies Algorithm 1.

The three rules each target a failure mode:

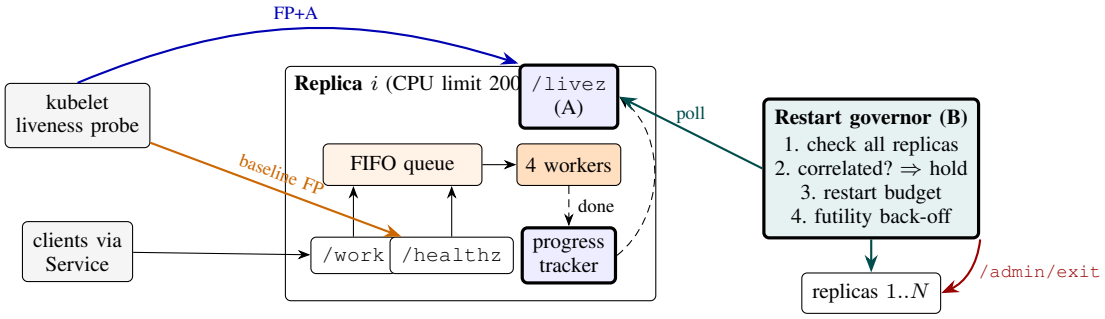


Fig. 3. Proposed method. Baseline FP probes `/healthz`, which waits in the same FIFO queue as real work. Component A answers `/livez` from a progress tracker outside the queue. Component B replaces the per-container restart decision with one decision over all replicas.

- The *correlation guard* (θ=0.5, H=60 s) treats many replicas failing together as a sign of a shared cause, which a restart will not fix.
- The *budget* (β=0.25) works like `maxUnavailable`: it never takes down more than a quarter of the replicas at once.
- The *futility back-off* lengthens the hold when restarts demonstrably do not help.

A single hung replica never trips the guard, so it is restarted as soon as it is a suspect.

The governor restarts a replica *in place* by calling its `/admin/exit` hook. The process exits and the kubelet restarts the container with its normal back-off, so a governor restart costs the same as a kubelet restart and the comparison is fair. If the hook does not answer, the governor deletes the pod.

The decision logic is a pure function (`src/internal/policy`) with unit tests, and the simulator mirrors it line by line. The controller is a 1-replica Deployment. Together with the policy it is about 400 lines and uses only the Go standard library.

D. How the components map to the gap

- A answers G2 (slow vs. dead) without asking the application to log its own failure, which SKI does require.
- B answers G3 (no aggregation), which is a limitation the paper names itself.
- Testing with $N = 6$ replicas, under load and with three kinds of stress answers G1.

E. Parameters and their trade-off

W is the only important parameter of A. A small W catches hangs fast but treats short stalls as hangs. Detection time is about $W + P$ (Sec. XI-H). We use W=5 s, which is 100 times the normal service time. B's parameters behave like rolling-update settings, and the results are not sensitive to θ (Table XII).

F. Overhead

A needs two atomic counters per request. B adds one pod and N small HTTP checks per second. On the testbed the governor used *pending* CPU-seconds and *pending* MiB per run.

X. Experimental Design

A. Arms

Every arm uses the same service, readiness probe, replicas and load; only the liveness mechanism changes.

- **NONE**: no liveness probe (control, from Cutoff 1).
- **DP**: the paper's default probe timing (3 s, 3 failures).
- **FP**: the paper's fast probe on `/healthz` (1 s, 1 failure). This is our **baseline**.
- **ISO**: FP timing on an isolated always-200 endpoint.
- **FP+A**: FP timing on `/livez` (Component A).
- **FP+B**: governor on `/healthz` (Component B).
- **FP+A+B**: governor on `/livez` (proposed method).

The four arms FP, FP+A, FP+B and FP+A+B form the 2×2 ablation the instructor asked for.

B. Scenarios

- **R**: the paper's reproduction (Sec. VI).
- **C**: calibration; measures the capacity of one replica and how probe latency grows with load.
- **O**: sustained load (no fault) at 50–150 % of capacity.
- **P**: a 60 s spike from 70 % to 150 %, then back to 70 %.
- **H**: one replica hangs silently at 60 s (a real failure that must be caught).
- **D**: the shared dependency stalls for 20 s (every replica stops at once; a restart cannot help).
- **S**: the Cutoff-1 grid of timeoutSeconds $\{1, 2, 5\}$ × failureThreshold $\{1, 3, 10\}$ at period 5 s.

C. Metrics

- **Success rate**: requests answered with 2xx within the 2 s client timeout.
- **Restarts**: container restarts during the load window.
- **Availability**: mean fraction of Ready replicas.
- **Cascade depth**: the largest number of replicas not Ready at the same time.
- **Post-spike success** (P): success rate after the spike ends.
- **Recovery time** (P, D): seconds from the end of the stress until per-second success stays $\geq 90\,\%$ for 5 s.
- **Detection time** (H): time from fault to the restart decision.
- **p95/p99 latency** of successful requests.
- **Overhead**: CPU and memory used.

D. Statistics

The simulator runs 10–30 seeds per cell. We report means and, where useful, standard deviations. Testbed cells have 1–5 repetitions, which is enough to confirm the direction and size of effects that the simulator shows clearly. Arms are interleaved, so slow drift of the machine affects all arms equally.

XI. Results

Unless stated otherwise, all numbers are means over seeds. "Sim" means simulator and "TB" means Kubernetes testbed.

A. Calibration: load turns into probe latency (C)

Before using Kubernetes we checked the coupling on *real processes* with `experiments/local_check.py`. One SUT process ran pinned to one core (native process pinned to 1 core, GOMAXPROCS=8), and our load generator measured it. The measured capacity was 270 req/s.

- At 50 % load, `/healthz` answered in 0 ms and `/livez` in 0.4 ms (medians).
- At 90 % load, `/healthz` rose to 28 ms, while `/livez` stayed at 3.3 ms.
- At 150 % load, 10/10 `/healthz` checks hit the 1 s probe timeout. `/livez` still answered in 3.0 ms.
- After a silent hang, `/livez` switched to 503 after 5.0 s ($= W$), as designed. The isolated check (ISO) never failed.

This is the gap mechanism in miniature. The baseline check fails because of load alone, while Component A does not.

One practical lesson came out of this test. If the Go runtime has only one scheduler thread (`GOMAXPROCS=1`), a saturated process delays even `/livez` beyond 1 s. In that case A would also give false positives. In a pod, Go 1.22 uses the host CPU count (8 on the HP 348), so the problem does not arise, but deployments must keep `GOMAXPROCS` above the pool size (`results/local/local_check_gomaxprocs1.json`).

On the testbed, the measured capacity of one replica with a 200m limit is *pending* req/s (Table V, Fig. 4).

B. Sustained load: probes manufacture failures (O)

Fig. 5 and Table VI answer RQ1 and RQ2.

- **Up to 80 % load**, all arms behave the same.
- **At 90 % load**, nothing is broken: NONE serves 100.0 % of requests. FP restarts containers 25 times, Ready replicas drop to 59 % and success falls to 58.9 %.
- **At 100 % load**, FP gives 13.8 % success and DP gives 33.0 %, against 90.9 % with no probe. *No liveness probe is strictly better than a standard one*, which was the headline we expected in Cutoff 1.
- **Component A** removes every false restart at every load: FP+A matches NONE.
- **Component B alone** reduces the damage (9.2 restarts and 68.7 % success at 100 %) but does not remove it. Its check is still the queued `/healthz`, and when the hold expires the budgeted restarts resume.

TABLE V
TB calibration

pending – run `experiments/02_calibrate.sh`

Figure appears after the testbed run.

Fig. 4. TB: health-check latency against load.

C. A short spike becomes a lasting outage (P)

Fig. 6 and Table VII show the most striking result. During the 60 s spike every arm drops requests, because the capacity is simply not there. When the load returns to 70 %, NONE recovers in about 2 s.

FP does *not* recover. Replicas restarted during the spike sit in CrashLoopBackOff, the survivors are overloaded, fail their probes and are restarted in turn. The service stays at 4.6 % success for the remaining two minutes, long after the trigger is gone. This is a *metastable failure* [13], [14] caused by the self-healing loop itself. DP recovers after about 29 s. FP+A and FP+A+B recover like NONE (99.3 %).

D. Real hangs are still caught (H)

A method that never restarts anything would "win" every overload test. Experiment H checks that real hangs are still caught. One replica hangs silently at 60 s (Table VIII).

- NONE and ISO *never* detect the hang. That replica stays useless and capacity is lost for good, which is the false-negative problem.
- SKI would behave like NONE, because a hung process writes no log line.
- FP and FP+B detect the hang in 1.5 s.
- FP+A and FP+A+B detect it in 5.4 s, roughly W.

This is the price of A: about 4 s slower detection, which buys immunity to overload. Success rates for all detecting arms stay above 99 %.

E. Shared brown-out: why B is needed (D)

In a dependency stall every replica stops completing work at the same time, so no single container is at fault. Here Component A alone is *not* enough. After W seconds without progress, every replica looks hung, and FP+A restarts 14.2 containers per run. FP is worse still, with 23.9 restarts and 66.0 % success.

The governor's correlation guard sees that six of six replicas fail together and holds the restarts. FP+A+B makes 0.0 restarts and reaches 92.2 % success, which equals NONE (92.2 %), the best possible result for a 20 s outage (Fig. 7, Table IX).

F. Ablation

Table X isolates each component.

- **A alone** fixes overload (O, P) but not shared stalls (D).
- **B alone** fixes shared stalls and most of the spike, but still restarts under sustained 90 % load.

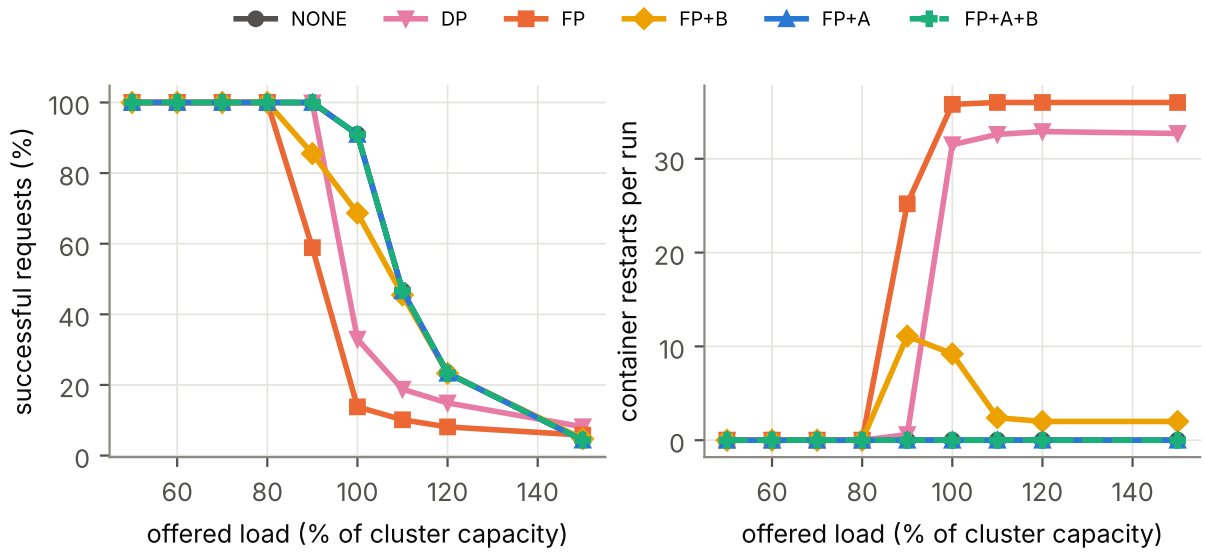


Fig. 5. Sim, sustained load (10 runs per point). Left: success rate. Right: restarts per 240 s run. FP and DP restart healthy containers from 90 % load. Component A (and A+B) removes every one of these false restarts.

TABLE VI
SUSTAINED LOAD (O): SUCCESS RATE (%) AND RESTARTS PER RUN

	60 %		90 %		100 %		120 %	
Arm (Sim)	Succ.	Restarts	Succ.	Restarts	Succ.	Restarts	Succ.	Restarts
NONE	100.0	0.0	100.0	0.0	90.9	0.0	23.4	0.0
DP	100.0	0.0	99.9	0.6	33.0	31.5	14.9	32.9
FP	100.0	0.0	58.9	25.2	13.8	35.8	8.1	36.0
ISO	100.0	0.0	100.0	0.0	91.0	0.0	23.3	0.0
FP+A	100.0	0.0	100.0	0.0	91.0	0.0	23.3	0.0
FP+B	100.0	0.0	85.5	11.1	68.7	9.2	23.3	2.0
FP+A+B	100.0	0.0	100.0	0.0	91.0	0.0	23.3	0.0

Testbed rows (90 % load) appear here after `experiments/03_overload.sh`.

TABLE VII
LOAD SPIKE TO 150 % (P)

Arm	Restarts	Ready %	Post-spike succ. %	Recovery s
Simulator (10 runs)				
NONE	0.0	100.0	99.3	2
DP	25.5	78.5	69.4	29
FP	30.1	35.3	4.6	never
ISO	0.0	100.0	99.3	2
FP+A	0.0	100.0	99.3	2
FP+B	1.7	99.6	99.0	5
FP+A+B	0.0	100.0	99.3	2
Testbed				
pending – run the testbed scripts (Appendix A)				

TABLE VIII
SILENT HANG OF ONE REPLICA (H, 70 % LOAD)

Arm	Detected	Detect. time s	Success	Ready %
Simulator (20 runs)				
NONE	0/20	never	0.960 ± 0.002	91.5
DP	20/20	8.5 ± 1.0	0.994 ± 0.001	99.8
FP	20/20	1.5 ± 0.3	0.999 ± 0.000	99.8
ISO	0/20	never	0.960 ± 0.002	91.5
FP+A	20/20	5.4 ± 0.3	0.996 ± 0.001	99.8
FP+B	20/20	1.5 ± 0.3	0.998 ± 0.000	99.8
FP+A+B	20/20	5.4 ± 0.3	0.996 ± 0.000	99.7
Testbed				
pending – run the testbed scripts (Appendix A)				

- **A+B** is the only combination that is best or tied-best in every column except hang-detection speed, where it pays the expected $\approx W$.

Each component is therefore responsible for a different part of the improvement.

G. Where is the boundary? (Cutoff-1 RQ1, S)

Fig. 8 sweeps the Cutoff-1 grid with `periodSeconds`=5. The probe causes harm (more than 1 point below the control) in 4 of 27 cells, and all of them are at 100 % load. Large harm (44–72 points) occurs only with `timeoutSeconds`=1 and `failureThreshold`$\leq$ 3. A 2 s timeout or a threshold of

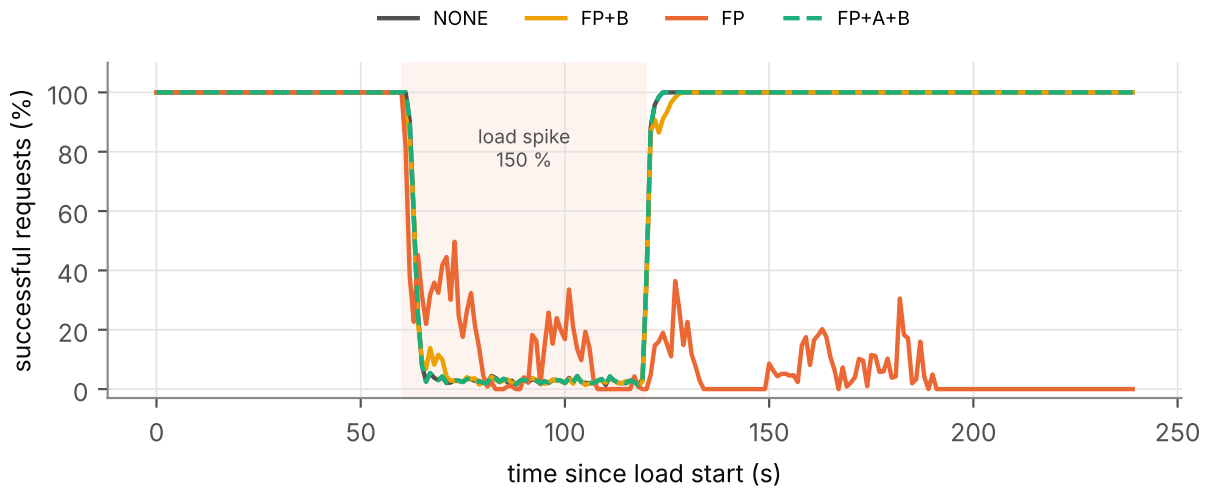


Fig. 6. Sim, load spike (mean of 10 runs). After the spike, FP stays down: the probe loop keeps the system in a bad state (metastable failure). The proposed A+B recovers at once.

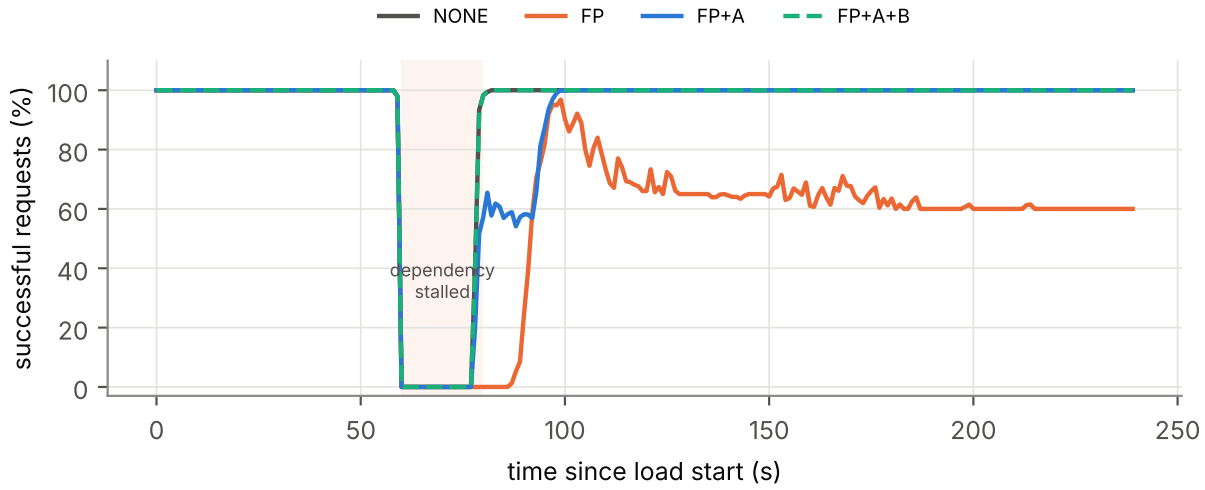


Fig. 7. Sim, 20 s dependency stall (mean of 20 runs). A alone restarts every replica once progress has stopped for W s. Only A+B recovers like NONE as soon as the dependency returns.

10 cuts the harm to a few points at this load.

At 50 % load the probe is harmless. At 150 % load the control has already collapsed, and restarts even help a little, because killing a container empties its queue of requests that are already doomed. The boundary is therefore a *band* around saturation, and it is narrower with a slow period (5 s) than with FP (1 s), where harm starts at 90 % (Fig. 5).

H. Sensitivity

Stall window W (Table XI, Fig. 9). Detection time grows as $W+0.3$ s. False restarts during a 20 s stall fall only once W approaches the stall length. No single W handles both hangs and long brown-outs, and this is exactly why B is needed: with B, W can stay small.

Overload never causes false restarts with A, whatever the value of W, because an overloaded replica always makes progress.

Governor (Table XII). With 6 of 6 replicas stalled, any $\theta \leq 0.67$ holds the restarts. A hold shorter than the stall (30 s against a 40 s stall) lets one budgeted restart through. Turning the guard off (budget only) gives 8 restarts.

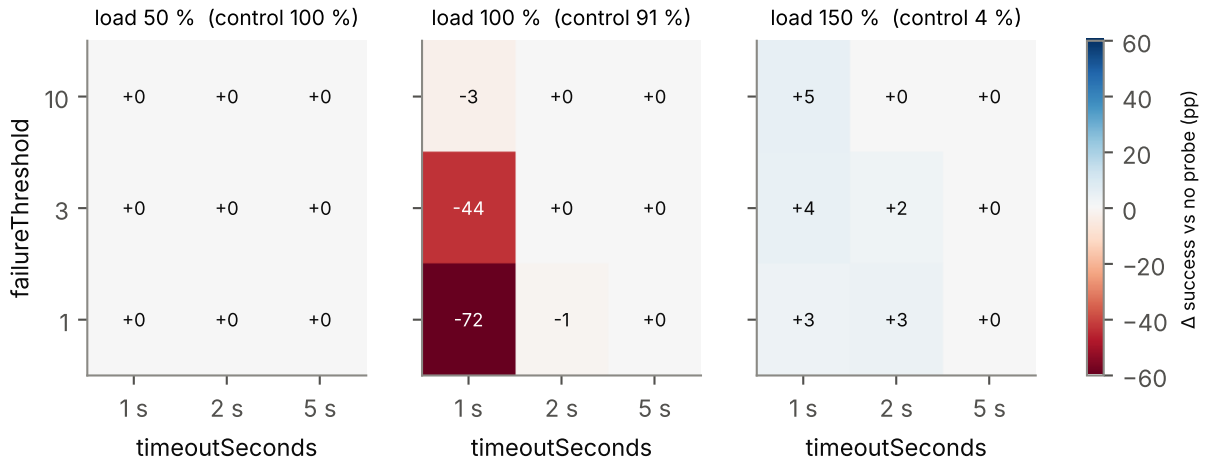


Fig. 8. Sim, Cutoff-1 sweep (period 5 s, 10 runs per cell): change in success rate against no probe, in percentage points. Red = the probe made things worse.

TABLE IX
DEPENDENCY STALL OF 20 S (D, 70 % LOAD)

Arm	Restarts	Success	Bad s	Recovery s
Simulator (20 runs)				
NONE	0.0 ± 0.0	0.922 ± 0.002	18.9	0.1
DP	7.3 ± 0.8	0.906 ± 0.003	22.2	4.2
FP	23.9 ± 5.7	0.660 ± 0.253	81.2	16.3 (15/20)
ISO	0.0 ± 0.0	0.922 ± 0.002	18.9	0.1
FP+A	14.2 ± 0.9	0.894 ± 0.022	24.9	12.3
FP+B	0.8 ± 0.7	0.920 ± 0.003	18.9	2.6
FP+A+B	0.0 ± 0.0	0.922 ± 0.002	18.9	0.1
Testbed				
pending – run the testbed scripts (Appendix A)				

"Bad s" = seconds with less than 50 % successful requests (20 are unavoidable). Recovery is measured from the end of the stall; "(k/n)" means only k of n runs recovered within the run.

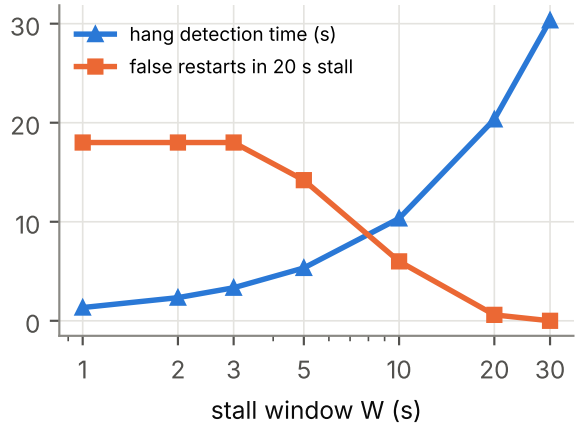


Fig. 9. Sim: the W trade-off for Component A alone.

TABLE X
ABLATION OVER THE FOUR STRESS SCENARIOS (HIGHER SUCCESS IS BETTER; FEWER RESTARTS AND FASTER DETECTION ARE BETTER)

			O: 90 % load		P: spike	H: hang	D: dependency stall	
System	A	B	Succ. %	Restarts	Post-spike succ. %	Detect. s	Restarts	Succ. %
Simulator								
FP			58.9	25.2	4.6	1.5	23.9	66.0
FP+A	✓		100.0	0.0	99.3	5.4	14.2	89.4
FP+B		✓	85.5	11.1	99.0	1.5	0.8	92.0
FP+A+B	✓	✓	100.0	0.0	99.3	5.4	0.0	92.2
Testbed								
pending – run the testbed scripts (Appendix A)								

TABLE XI
SIM: EFFECT OF THE STALL WINDOW W (A ONLY)

W (s)	H: detect. s	D: restarts	O 100 %: restarts
1	1.3	18.0	0.0
2	2.3	18.0	0.0
3	3.3	18.0	0.0
5	5.3	14.2	0.0
10	10.3	6.0	0.0
20	20.3	0.6	0.0
30	30.3	0.0	0.0

TABLE XII
SIM: GOVERNOR PARAMETERS (A+B, 40 S STALL / HANG)

θ	H (s)	D: restarts	D: succ. %	H: detect. s
0.34	30	1.0	83.7	5.5
0.34	60	0.0	83.9	5.5
0.34	120	0.0	83.9	5.5
0.50	30	1.0	83.7	5.5
0.50	60	0.0	83.9	5.5
0.50	120	0.0	83.9	5.5
0.67	30	1.0	83.7	5.5
0.67	60	0.0	83.9	5.5
0.67	120	0.0	83.9	5.5
off	30	8.0	82.6	5.5
off	60	8.0	82.6	5.5
off	120	8.0	82.6	5.5

XII. Final Comparison and Discussion

Table XIII brings everything together.

What was reproduced? The detection-time model and the DP/FP/SKI detection times, within about 1 s. The ranking and size of the availability differences. The FP extra-restart anomaly, which we also explain.

What could not be reproduced? SKI's exact mechanism, because the modified kubelet is not public; we reproduce its behaviour instead. The paper's total and failed request counts, which depend on undocumented Locust settings. The "catalogue bug". And 30 repetitions on real hardware, where we run 3–5.

Why are there differences? The client model, container runtime, hardware, start-up time and the paper's own one-second model error (Sec. VI-C).

What gap was identified? Monitoring that confuses slow with dead and decides per container, which a single-pod evaluation cannot expose (Sec. VIII).

How does the proposal address it? A judges progress, not latency. B decides across replicas.

What is the evidence? The 2×2 ablation over four stress scenarios (Table X); the replica-count experiment (Fig. 2); and real-process measurements of the latency–load coupling. Testbed confirmation is in the TB rows.

Remaining limitations. A needs two hooks in the application, and its check needs a free scheduler thread (`GOMAXPROCS` above the pool size). This is a smaller change than SKI's log contract, but it is still a change. Detection of silent hangs is slower by about W. If many replicas hang at the *same* time from a bug inside each pod, B delays the needed restarts by up to H. The futility rule and the hold bound this delay, but do not remove it.

TABLE XIII
FINAL COMPARISON: ORIGINAL PAPER, REPRODUCED BASELINE AND PROPOSED METHOD

	Original paper [1]	Reproduced baseline (FP)	Proposed (FP+A+B)
Setting	1 pod, $\leq$100 req/s	1 pod (R) and 6 pods (O, P, H, D)	6 pods
Process crash: time to queue restart	0.7 s (FP), 0.1 s (SKI)	0.8 s (sim) / *pending* s (TB)	not re-measured (a dead process refuses the connection, so the check fails at once)
Silent hang: detection	not tested	1.5 s	5.4 s ($\approx W$)
90 % load, no fault: success	not tested	58.9 % (25 restarts)	100 % (0 restarts)
After 60 s spike: success	not tested	4.6 %	99.3 %
20 s dependency stall: success	not tested	66.0 %	92.2 %
Extra runtime cost	SKI: +0.04 vCPU, +9 MiB per pod	–	2 counters/request; 1 governor pod (*pending* MiB)
Needs modified kubelet?	Yes	No	No
Needs app to log its failure?	Yes (SKI)	No	No (needs `Begin/End` hooks)

XIII. THREATS TO VALIDITY

Simulation is a model. Processor sharing ignores the 100 ms CFS quota periods. The simulator also assumes that `/livez` answers instantly; the real-process check shows this holds only when the runtime has spare scheduler threads. Endpoint and back-off timing are simplified from the Kubernetes source. We reduce this threat in three ways: by reproducing the paper's numbers first, by checking the coupling on real processes, and by running every scenario on the real testbed.

Single host. CPU limits make each pod saturate before the laptop does. The calibration reports the *measured* capacity.

Synthetic service. Our SUT is minimal on purpose. The mechanism that generalises is a health check that shares resources with work. `catalogue`'s `/health` has the same property through its database.

Few testbed repetitions. Effects of tens of percentage points are visible in 1–5 runs. Small differences are judged only on the simulator.

XIV. POINTS FOR MENTOR DISCUSSION AND NEXT STEPS

1) Is reproducing SKI's behaviour on stock Kubernetes acceptable, given that the authors' modified kubelet is private?
2) Is the gap (G1–G3) accepted as the direction for Cutoff 3?
3) For Cutoff 3 we plan to:
   - run the *full* testbed profile;
   - add a multi-service benchmark (TeaStore or the full SockShop);
   - add `catalogue` under load as a second, unmodified application;
   - compare against an adaptive timeout (a φ-accrual-style detector [12]).

XV. CONCLUSION

We re-implemented the core of Roberts *et al.* on stock Kubernetes and in a calibrated simulator, and reproduced its detection times. We also explained its unexplained restarts.

The reproduction exposed a gap. The paper's single-pod, light-load setting hides the fact that poll-based self-healing *creates* outages in replicated services under load. Signal-based monitoring fixes only part of this.

Our method has two parts. Progress-based liveness tells slow from dead. A cluster-aware governor refuses to restart replicas that fail together. Together they remove every false restart in our overload, spike and brown-out scenarios, and they still catch real hangs.

APPENDIX A
HOW TO REPRODUCE EVERYTHING

```
# simulator (any laptop, ~5 min, no Docker needed)
python3 sim/run_all.py
# Kubernetes testbed (~5 h, see docs/GUIDE.md)
PROFILE=quick bash experiments/run_all.sh
python3 analysis/analyze_testbed.py
# rebuild this PDF (testbed tables fill in)
cd report && latexmk -pdf main.tex
```

Repository layout: `src/` (Go: SUT, load generator, SKI emulation, governor), `manifests/`, `cluster/`, `experiments/`, `sim/`, `analysis/`, `results/`, `report/`, `docs/`.

REFERENCES

[1] J. Roberts, B. Archibald, and P. Trinder, "Signalling health for improved kubernetes microservice availability," *arXiv preprint arXiv:2507.02158*, 2025, university of Glasgow. [Online]. Available: https://arxiv.org/abs/2507.02158

[2] The Kubernetes Authors, "Configure liveness, readiness and startup probes," Kubernetes documentation, accessed Oct. 2026. [Online]. Available: https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/

[3] Weaveworks, "Sock shop: A microservices demo application," GitHub repository, images weaveworksdemos/catalogue:0.3.5 and catalogue-db:0.3.0. [Online]. Available: https://github.com/microservices-demo/microservices-demo

[4] Locust developers, "Locust: An open source load testing tool." [Online]. Available: https://locust.io/

[5] J. Flora, P. Gonçalves, M. Teixeira, and N. Antunes, "A study on the aging and fault tolerance of microservices in Kubernetes," *IEEE Access*, vol. 10, pp. 132 786–132 799, 2022.

[6] Z. Chen, M. Goudarzi, and A. N. Toosi, "Resilience evaluation of Kubernetes in cloud-edge environments via failure injection," arXiv preprint arXiv:2507.16109, 2025.

[7] L. A. Vayghan, M. A. Saied, M. Toeroe, and F. Khendek, "Kubernetes as an availability manager for microservice applications," arXiv preprint arXiv:1901.04946, 2019.

[8] Kubernetes SIGs, "kind: Kubernetes in docker, release v0.23.0," GitHub repository, node image kindest/node:v1.27.13. [Online]. Available: https://github.com/kubernetes-sigs/kind

[9] The Kubernetes Authors, "Pod readiness gates," Kubernetes documentation (Pod Lifecycle), accessed Oct. 2026. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/#pod-readiness-gate

[10] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016, chapters 21–22: Handling Overload; Addressing Cascading Failures.

[11] T. D. Chandra and S. Toueg, "Unreliable failure detectors for reliable distributed systems," *Journal of the ACM*, vol. 43, no. 2, pp. 225–267, 1996.

[12] N. Hayashibara, X. Défago, R. Yared, and T. Katayama, "The φ accrual failure detector," in *Proceedings of the 23rd IEEE International Symposium on Reliable Distributed Systems (SRDS)*, 2004, pp. 66–78.

[13] N. Bronson, A. Aghayev, A. Charapko, and T. Zhu, "Metastable failures in distributed systems," in *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS)*, 2021, pp. 221–227.

[14] L. Huang, M. Magnusson, A. B. Muralikrishna, S. Estyak, R. Isaacs, A. Aghayev, T. Zhu, and A. Charapko, "Metastable failures in the wild," in *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022, pp. 73–90.